Unit Testing 1

To do: Make a submission **To do:** Receive a grade

Due: Tuesday, 24 September 2024, 10:00 AM

Overview

In this assignment, you will write unit tests for a number guessing game application. The game involves choosing a difficulty level, placing bets, and guessing numbers. The top ten high scores are saved to a scoreboard.

You will start by answering conceptual questions related to equivalence partitioning, boundary value analysis, robustness testing, and decision table testing. Then, you will implement unit tests for the application using these concepts. Finally, you will write comprehensive tests for all functions in `game.py`.

Start by downloading the game source code, which is attached to this assignment (see below).

Game Rules

- You start with 100 bet points.
- Choose a difficulty level:
  - Easy: 10 guesses
  - Hard: 5 guesses
- Place a bet with your bet points for each round.
- Guess the number between 1 and 100:
  - If you guess correctly, you can choose to double your prize or take your winnings.
  - If you choose to double, you have a 50% chance to either double your prize or lose it.
- If you run out of guesses, you can try to save your bet:
  - You have a 50% chance to keep your bet points or lose the bet points you placed for that round.
- After each round, you can decide to continue playing or finish the game and save your score to the scoreboard.
- The game ends when you choose to finish or you have no bet points left.
- The high scores are saved and displayed on the scoreboard. Try to get your name in the top 10!

How to Play the Game

Web Version:

- Run the Web Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python app.py`
  - Open your web browser and go to `http://127.0.0.1:5000/`.

CLI Version:

- Run the CLI Application:
  - Open a terminal and navigate to the project directory.
  - Run the following command: `python game.py`
  - Follow the prompts:
    - Enter your name.
    - Choose a difficulty level.
    - Place bets and guess numbers as prompted in the terminal.

How to Install Dependencies

- Ensure you have Python installed on your system.
- Open a terminal and navigate to the project directory.
- Run the following command to install the required dependencies:
  `pip install -r requirements.txt`

How to Write and Test Unit Tests

There are test files named `test_theoretical_concepts.py`, and `test_game.py`.

Implement Test Cases:

- Follow the function names and TODO comments provided below to implement your test cases.

Conceptual Questions

- **Equivalence Partitioning:** Explain how equivalence partitioning can be applied to the number guessing functionality. Identify at least three equivalence classes for valid and invalid guesses.
- **Boundary Value Analysis (BVA):** Discuss how boundary value analysis can be used to test the bet placing functionality. Identify the boundary values you would test.
- **Robustness Testing:** Describe how robustness testing can be applied to the difficulty selection functionality. What are the expected valid and invalid inputs?
- **Decision Table Testing:** Create a decision table to cover the scenarios where the user decides to double their winnings or try to save their bet. Include all possible outcomes.

File Structures of the Assignment:

- **NUMBER-GUESSING-GAME**
  - **templates**
    - bet.html
    - correct.html
    - difficulty.html
    - end.html
    - guess.html
    - index.html
    - out_of_guesses.html
  - **tests**
    - test_game.py
    - test_theoretical_concepts.py
  - app.py
  - game.py
  - routes.py
  - scoreboard.py
  - conceptual_answers.txt
  - scoreboard.txt
  - requirements.txt

Note:

In your tests, you will use `@patch` to mock certain functions. This is particularly useful for functions that involve randomness or external dependencies. For instance, you will use `@patch` to mock the `random.choice` function in the `double_prize` and `save_bet` methods to control the outcome of these methods and test different scenarios effectively.

Implementation of Unit Tests

You will implement unit tests in these separate files:

- **test_theoretical_concepts.py:** Tests for equivalence partitioning, boundary value analysis, robustness testing, and decision table testing.
- **test_game.py:** Tests for all functions in `game.py`.

Explanations for the tests:

1. test_theoretical_concepts.py:

- **test_valid_guesses:** Test valid guesses by setting the target number and checking guesses within the valid range.
- **test_invalid_guesses:** Test invalid guesses by checking guesses outside the valid range and non-numeric inputs.
- **test_boundary_values:** Test boundary values by setting bet points and checking bets at boundary values and invalid values in the beginning of the game.
- **test_boundary_values_after_rounds:** Test boundary values after rounds by setting bet points, target number, and current bet, and checking bets at boundary values and invalid values when the player has 320 points.
- **test_robustness_difficulty:** Test robustness by checking setting difficulty with valid and invalid inputs.
- **test_decision_table_double_success:** Test `double_prize` method with success by setting the current bet and bet points.
- **test_decision_table_double_failure:** Test `double_prize` method with failure by setting the current bet and bet points.
- **test_decision_table_no_double:** Test `double_prize` method when the 'no' option is selected by setting the current bet and bet points.
- **test_decision_table_save_success:** Test `save_bet` method with success by setting the current bet and bet points.
- **test_decision_table_save_failure:** Test `save_bet` method with failure by setting the current bet and bet points.
- **test_decision_table_no_save:** Test `save_bet` method when the 'no' option is selected by setting the current bet and bet points.

2. test_game.py

- **test_initial_state:** Test the initial state of the game. Ensure all initial attributes are correctly set.
- **test_set_name:** Test setting the name for the game. Ensure a `ValueError` is raised for an empty name.
- **test_set_difficulty:** Test setting the difficulty for the game. Ensure the max guesses are set correctly and a `ValueError` is raised for an invalid difficulty.
- **test_place_bet:** Test placing a bet in the game. Ensure the bet is set correctly and `ValueError` is raised for invalid bets.
- **test_reset_round:** Test resetting the game round. Ensure the max guesses and target number are reset correctly.
- **test_guess_number_correct:** Test guessing the correct number. Ensure the method returns the correct result when the guess is correct.
- **test_guess_number_higher_lower:** Test guessing a number higher or lower than the target. Ensure the method returns the correct result and the number of guesses decreases.
- **test_guess_number_out_of_guesses:** Test the scenario where the player runs out of guesses. Ensure the method returns the correct result when guesses are exhausted.
- **test_double_prize_success:** Test doubling the prize successfully. Ensure the method returns the correct result when doubling is successful.
- **test_double_prize_failure:** Test failing to double the prize. Ensure the method returns the correct result when doubling fails.
- **test_save_bet_success:** Test saving the bet successfully. Ensure the method returns the correct result when saving is successful.
- **test_save_bet_failure:** Test failing to save the bet. Ensure the method returns the correct result when saving fails.
- **test_check_goal:** Test checking the game goal (lose/not lose conditions). Ensure the method returns the correct result based on the game's current state.

Running Tests with `unittest`

To avoid import issues when running your tests, you can use the following command:

```
python -m unittest tests.test_game
```

The `tests` directory is recognized without modifying `sys.path` or moving files. If you want to run all the test files in the `tests` folder, you can use the command:

```
python -m unittest discover -s tests
```

Submission: File Submission, i.e., upload the whole project folder as a zip file. Double check that your zip file includes updated test files and the conceptual_answers.txt.

Grading:

- 5-4 points: The assignment is clear and insightful, with correct implementation of test cases and well-explained theory. Test cases are complete, and edge cases are covered.
- 3-2 points: The assignment is mostly clear but may lack some critical thinking. Test cases are implemented, but some important edge cases are missed.
- 1-0 points: The assignment lacks clarity or coverage of critical concepts. Test cases are missing or fail to cover key functions or edge cases.

Use of AI and Collaboration with Other Students: Using AI tools to produce an answer for this assignment is not permitted.All parts of the submission must be produced by yourself (apart from files provided in the assignment). Collaboration with other students is not permitted. This is an individual assignment that you must author independently.

UnitTesting1.zip 8 September 2024, 2:34 PM

Submission status

Attempt number	This is attempt 1.
Submission status	No submissions have been made yet
Grading status	Not marked
Time remaining	Assignment is overdue by: 169 days 4 hours
Last modified	-